Licence de Sciences et Technologies
S3 – mention Informatique

Université de Perpignan Via Domitia
V. Zucca

Programmation en C

TP2 : Fonctions et pointeurs

Exercice 1. On considère un triplet $(a, b, c) \in \mathbb{R}^3$ et l'équation du second degré associée

$$ax^2 + bx + c = 0$$

1. Écrire une fonction `solve_quadratic` qui prend en paramètre les coefficients a, b et c et calcule les éventuelles solutions de l'équation du second degré associée.
2. Trouver un moyen permettant à la fonction de renvoyer les éventuelles solutions.

Exercice 2. On considère le code suivant

```c
#include <stdio.h>

void analyser_temperature(double temperature, double *resultat)
{
    double celsius;
    double fahrenheit;

    celsius = temperature;
    fahrenheit = temperature * 9.0 / 5.0 + 32.0;

    if (temperature > 0)
        resultat = &celsius;
    else
        resultat = &fahrenheit;
}

void afficher(double valeur)
{
    printf("Valeur sélectionnée : %.2f\n", valeur);
}

int main(void)
{
    double resultat;

    analyser_temperature(-12.5, &resultat);

    afficher(resultat);

    return 0;
}
```

1. Quelle valeur le programme ci-dessus est-il censé afficher ?
2. Quel est le problème ?
3. Proposer deux façons différentes de corriger le problème.

Exercice 3. On considère le code ci-dessous

```c
#include <stdio.h>

int main()
{
  int *n;
  printf("Entrez une valeur pour n : ");
  scanf("%d",n);

  printf("Vous avez entré la valeur n = %d\n",*n);

  return 0;
}
```

1. Que se passe-t-il à l'exécution du code ci-dessus ?
2. D'où vient le problème et comment le corriger ?

Exercice 4. On souhaite coder une fonction `make_change` qui indique la quantité d'argent à rendre lors d'un paiement.

1. Coder une première version de la fonction ayant la signature suivante :

```c
float make_change(float price, float amount);
```

2. On souhaite maintenant que la fonction indique le nombre de pièces de 2€, 1€, 50 centimes, 20 centimes, 10 centimes, 5 centimes, 2 centimes et 1 centime qu'elle doit rendre pour faire l'appoint.
   (a) Proposer un nouvel entête permettant d'atteindre le but recherché.
   (b) Coder le corps de la fonction
3. Écrire un programme demandant à l'utilisateur le prix à payer, le montant reçu et qui affiche le nombre de pièces à rendre pour rendre la monnaie.

   Exemple d'exécution :

```
Indiquez le prix à payer : 123.58
Indiquez le montant reçu : 130

Vous devez rendre 6.42 euros soit :
3 piece(s) de 2 euros
2 pièce(s) de 20 centimes
1 pièce(s) de 2 centimes
```

Exercice 5. On souhaite coder une fonction

```c
void print_base(unsigned int n, unsigned int b);
```

permettant d'afficher la représentation d'un entier positif `n` en base $2 \leq$ `b` ≤ 10. Pour cela, on rappelle que la division entière de n par b permet d'obtenir le chiffre de poids faible de n en base b.

$$n = b \left\lfloor \frac{n}{b} \right\rfloor + r \tag{1}$$

avec $0 \leq r < b$.

1. Utiliser l'équation (1) pour coder récursivement la fonction `print_base`.

   Exemples d'exécution :

```
Entrez un nombre n : 13
Entrez la base b : 2
Voici 13 en base 2 : 1101  //  13 = 1*8 + 1*4 + 1

Entrez un nombre n : 13
Entrez la base b : 3
Voici 13 en base 3 : 111   //  13 = 1*9 + 1*3 + 1


Entrez un nombre n : 13
Entrez la base b : 5
Voici 13 en base 5 : 23    //  13 = 2*5 + 3
```

2. Faire en sorte que la fonction préfixe son affichage par `b:`

   Exemples d'exécution :

```
Entrez un nombre n : 13
Entrez la base b : 2
13 = 2:1101

Entrez un nombre n : 13
Entrez la base b : 3
13 = 3:111
```

3. En utilisant les caractères `a-f` pour les chiffres de 10 à 15, faire en sorte que la fonction `print_base` gère la base 16.

Exercice 6. Dans cet exercice, on souhaite approcher numériquement l'unique solution α de l'équation

$$x^3 - 2x - 5 = 0$$

On considère la fonction

$$f : \left| \begin{array}{ccc} \mathbb{R} & \to & \mathbb{R} \\ x & \mapsto & x^3 - 2x - 5 \end{array} \right.$$

1. Calculer $f(2)$ et $f(3)$. Qu'en déduisez-vous sur α ?
2. Coder une fonction `double binary_search(double eps);` qui procède par dichotomie pour renvoyer une valeur $x \in \mathbb{R}$ telle que $|f(x)| <$ `eps`.
3. Modifier la fonction précédente pour récupérer également le nombre d'itérations nécessaires à l'obtention du résultat.

La méthode de Newton est un autre algorithme itératif permettant d'approcher numériquement les solutions réelles d'une équation $f(x) = 0$, où $f : \mathbb{R} \to \mathbb{R}$ dérivable sur $\mathbb{R}$.

L'algorithme suppose que la fonction f peut être approchée localement par sa tangeante. On rappelle que la tangeante de f en $x_0 \in \mathbb{R}$ est la droite d'équation :

$$y = f(x_0) + f'(x_0)(x - x_0)$$

Autrement dit, en choisissant x_0 proche de s, l'algorithme suppose que pour tout x suffisamment proche de x_0 on a

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0)$$

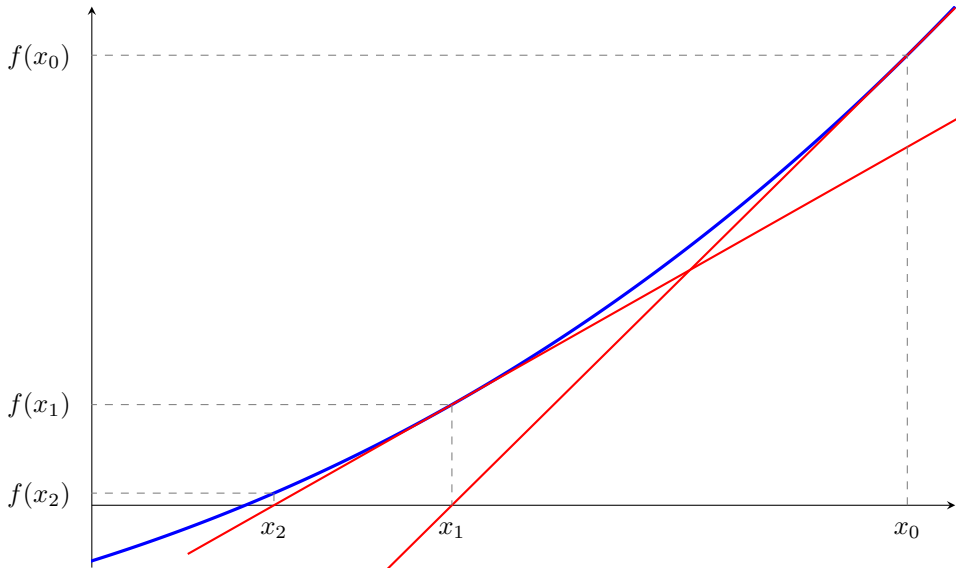
Or, si $f'(x_0) \neq 0$, la tangeante de f en x_0 coupe l'axe des abscisses en

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

On peut montrer que, sous certaines conditions, x_1 est plus proche de α que x_0. Ainsi en itérant le processus on peut montrer que la suite $(x_n)_{n\in\mathbb{N}}$ définie par

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

converge vers α.



4. De la même façon que pour `binary_search`, coder une fonction `newton_search` qui utilise la méthode de Newton pour renvoyer une approximation de α à `eps` près. La fonction prendra comme point de départ un `double` x_0.

5. Laquelle des deux fonctions est la plus efficace ?

6. Proposer une version récursive des deux fonctions.